Absolutely! Let's cover **File Handling in Python** from a **basic level**, step by step, with examples, exercises, and real-world applications.

Python File Handling – Basics

1️⃣ What is File Handling?

File handling in Python means **reading from and writing to files**.

- Files are used to **store data permanently** on your computer.
- Python provides built-in functions to work with files.

Common file types: .txt, .csv, .json, .py, etc.

2️⃣ Opening a File

To work with a file, first **open it** using the open() function.

file = open("example.txt", "mode")

Modes of File Handling

Mode	Meaning	Example Use
'r'	Read (default)	Open file to read data
'w'	Write	Open file to write (overwrites if exists)
'a'	Append	Add data at the end of file
'x'	Create	Create file, fails if exists
'r+'	Read & Write	Open file to read and write

Example: Opening a File for Reading

```
file = open("example.txt", "r")

print(file)  # <_io.TextIOWrapper ...>
```

```
file.close()  # Always close file after use
```

3️⃣ Reading from a File

3.1 Read Entire File

```
file = open("example.txt", "r")

content = file.read()  # Reads whole file

print(content)

file.close()
```

3.2 Read Line by Line

```
file = open("example.txt", "r")

print(file.readline())  # Reads first line

print(file.readline())  # Reads second line

file.close()
```

3.3 Read All Lines as a List

```
file = open("example.txt", "r")

lines = file.readlines()  # Returns a list of lines

print(lines)

file.close()
```

4️⃣ Writing to a File

4.1 Write Mode 'w' (Overwrite)

```
file = open("example.txt", "w")

file.write("Hello, Python!\n")

file.write("File handling is easy.\n")

file.close()
```

Note: 'w' overwrites the file if it already exists.

4.2 Append Mode 'a'

```
file = open("example.txt", "a")

file.write("This line is added at the end.\n")

file.close()
```

5️⃣ Using with Statement (Recommended)

The with statement **automatically closes the file** after the block ends.

```
with open("example.txt", "r") as file:

    content = file.read()

    print(content)


with open("example.txt", "a") as file:

    file.write("Using with statement is better!\n")
```

6️⃣ Deleting a File

```
import os


os.remove("example.txt")  # Deletes the file permanently
```

7️⃣ File Handling Example (Complete)

```
# Writing to a file

with open("students.txt", "w") as f:

    f.write("Ali, 20\n")

    f.write("Sara, 22\n")


# Appending data
```

```
with open("students.txt", "a") as f:
    f.write("Ahmed, 21\n")

# Reading data
with open("students.txt", "r") as f:
    data = f.read()
    print(data)
```

Output:

```
Ali, 20
Sara, 22
Ahmed, 21
```

8️⃣ Exercises – Basic Level

1. Create a file called notes.txt and write 5 lines of your favorite quotes.
2. Read the file and print each line separately.
3. Append 2 more quotes to the same file.
4. Count the number of lines in the file.
5. Delete a file called old_notes.txt (create it first if needed).

9️⃣ Real-World Applications of File Handling

- Storing user data for applications (e.g., username, passwords).
- Logging system errors in .log files.
- Reading configuration files like .ini or .json.
- Saving and loading CSV files for data analysis.
- Temporary storage for programs (cache).

If you want, I can make a **visual step-by-step diagram of File Handling in Python** showing **Opening → Reading/Writing → Closing → Deleting**. This makes it **very easy to remember**.

Do you want me to make that diagram?

Perfect! Let's expand **Python file handling** to include **Excel and Word files** along with the basics. I'll break it down step by step from **basic to intermediate**, with examples and exercises.

Python File Handling – Text, Excel, and Word

1️⃣ Basic Text File Handling Recap

Create, write, read, append, and delete text files:

```
# Writing to a text file
with open("example.txt", "w") as f:
    f.write("Hello, Python!\n")
    f.write("File handling is easy.\n")

# Reading from a text file
with open("example.txt", "r") as f:
    content = f.read()
    print(content)

# Appending to a file
with open("example.txt", "a") as f:
    f.write("This is an extra line.\n")

# Delete a file
import os
```

```
os.remove("example.txt")
```

2️⃣ Excel File Handling (.xlsx)

To work with Excel files, we use the **openpyxl** library.

Step 1: Install openpyxl

```
pip install openpyxl
```

Step 2: Create an Excel File

```
from openpyxl import Workbook

# Create a workbook and sheet
wb = Workbook()
sheet = wb.active
sheet.title = "Students"

# Add data
sheet["A1"] = "Name"
sheet["B1"] = "Age"
sheet.append(["Ali", 20])
sheet.append(["Sara", 22])
sheet.append(["Ahmed", 21])

# Save the file
wb.save("students.xlsx")
```

Step 3: Read Excel File

```
from openpyxl import load_workbook
```

```
wb = load_workbook("students.xlsx")
sheet = wb["Students"]

# Read data row by row
for row in sheet.iter_rows(values_only=True):
    print(row)
```

Output:

```
('Name', 'Age')
('Ali', 20)
('Sara', 22)
('Ahmed', 21)
```

Step 4: Update Excel File

```
wb = load_workbook("students.xlsx")
sheet = wb.active

sheet["C1"] = "Grade"
sheet["C2"] = "A"
sheet["C3"] = "B+"
sheet["C4"] = "A-"

wb.save("students.xlsx")
```

3️⃣ Word File Handling (.docx)

To work with Word files, we use the **python-docx** library.

Step 1: Install python-docx

```
pip install python-docx
```

Step 2: Create a Word Document

```
from docx import Document

doc = Document()
doc.add_heading("Python File Handling", 0)

doc.add_paragraph("This is an example of writing to a Word file.")
doc.add_paragraph("We can also add multiple paragraphs.")

doc.save("example.docx")
```

Step 3: Read Word Document

```
from docx import Document

doc = Document("example.docx")

for para in doc.paragraphs:
    print(para.text)
```

Output:

```
Python File Handling
This is an example of writing to a Word file.
We can also add multiple paragraphs.
```

Step 4: Add Tables to Word

```
from docx import Document

doc = Document()
table = doc.add_table(rows=1, cols=2)

# Add headers
hdr_cells = table.rows[0].cells
hdr_cells[0].text = 'Name'
hdr_cells[1].text = 'Age'

# Add rows
row_cells = table.add_row().cells
row_cells[0].text = 'Ali'
row_cells[1].text = '20'

row_cells = table.add_row().cells
row_cells[0].text = 'Sara'
row_cells[1].text = '22'

doc.save("students.docx")
```

4️⃣ Summary Table

File Type	Library / Method	Basic Operations
Text (.txt)	Built-in open()	read, write, append, delete
Excel (.xlsx)	openpyxl	create, read, update, add rows/columns

File Type	Library / Method	Basic Operations
Word (.docx)	python-docx	create, read, add paragraphs, tables

---

5️⃣ Exercises – Hands-on Practice

1. **Text File:**
   - Create a tasks.txt file and write 5 tasks.
   - Append 2 more tasks.
   - Read and print all tasks line by line.
2. **Excel File:**
   - Create grades.xlsx with columns: Student, Math, Science.
   - Add 3 students with marks.
   - Update one student's marks.
   - Read and print all data.
3. **Word File:**
   - Create report.docx with a heading.
   - Add 3 paragraphs describing your project.
   - Add a table with Name and Score for 3 students.
   - Read and print all paragraphs.

If you want, I can **create a visual diagram showing file handling for Text, Excel, and Word** in Python, with **step-by-step flow**. This makes it very easy to understand and remember.

Do you want me to make that diagram?